

LangChain Complete Series

Hinglish Study Notes

Videos 1–5 | CampusX | Instructor: Nitesh

■ Video 1	Introduction to LangChain — What & Why
■ Video 2	LangChain Components — 6 Core Parts
■ Video 3	Models Component — Language & Embedding Models
■ Video 4	Prompts Component — Static, Dynamic, Templates
■ Video 5	Structured Output — JSON, Pydantic, Schemas

■ Tagline: "If you want to study WHAT is X, you should first know WHY X was needed in the first place." — Instructor Nitesh

■ VIDEO 1 — Introduction to LangChain

■ What is LangChain?

LangChain (Definition)

LangChain is an **open-source framework** for developing applications **powered by LLMs** (Large Language Models). Agar aap LLM-based koi bhi application banana chahte ho to us application ko banane mein aapki help jo framework karta hai wo LangChain hai.

■ Real-World Example: "Chat with Your PDF" App (2014 Idea)

Problem Scenario: Around 2014, instructor ke dimag mein ek idea aaya — ek aisi application banao jahan koi bhi apna PDF upload kar sake, use read kar sake, *aur saath mein us PDF se directly baat bhi kar sake!* For example: "Page 5 explain karo jaise main 5 saal ka bachcha hun" ya "Linear Regression ke upar True/False questions generate karo."

■■ High-Level System Design of PDF Chat App:

Step 1 — PDF Upload	User apna PDF upload karta hai → system usse database mein store kar leta hai
Step 2 — User Query	User ek question puchta hai, jaise: "What are the assumptions of Linear Regression?"
Step 3 — Semantic Search	System puri book mein semantic search karta hai → relevant pages (e.g., Page 37, Page 46) milte hain
Step 4 — System Query Build	Retrieved pages + original user query ko milake ek system query banti hai
Step 5 — LLM (Brain)	Yeh system query LLM ke paas jaati hai jo query ko samajhta hai aur context-aware response deta hai
Step 6 — Final Output	LLM ka response user ko dikhaya jaata hai

■ Why Semantic Search (Not Keyword Search)?

- Keyword Search = words ko as-is search karo
- Problem: "assumptions" word bahut jagah aata hai
- Irrelevant pages bhi result mein aa jaate hain
- Inefficient, low precision

- Semantic Search = query ka MEANING samjho
- Sirf relevant pages milte hain
- "Assumptions of Linear Regression" → directly relevant pages
- Efficient, high precision, context-aware

■ How Semantic Search Actually Works (Embeddings):

- 1■■ Har paragraph/document ko **embedding (vector)** mein convert karo — numbers ka ek set jo text ka semantic meaning represent karta hai
- 2■■ Techniques: Word2Vec, Doc2Vec, BERT Embeddings, etc.
- 3■■ Example: 100-dimensional vector space mein Virat Kohli paragraph alag point pe hoga, Bumrah paragraph alag pe
- 4■■ User ki query ka bhi embedding banao (usi 100-dimension space mein)
- 5■■ Query vector ka teeno paragraph vectors ke saath **cosine similarity** nikalo
- 6■■ Jis paragraph ke saath highest similarity → wo relevant document hai → use answer ke liye use karo

■■ Detailed System Architecture (Low-Level Design):

Component	Role	Technology
AWS S3	PDF documents store karna	Cloud Storage
Text Splitter	PDF ko chunks mein tod na — splitter decide karta hai splitting ka tarika	NLP Model
Embedding Model	Text chunks ko vectors mein convert karna	BERT / OpenAI Embeddings
Vector Database	Embeddings ko store karna (permanent storage)	Pinecone, FAISS, ChromaDB
LLM (Brain)	Query samajhna + context-aware response generate karna (NLU + Text Generation)	GPT-4, Claude, Gemini

■■ Why NOT send the whole book to LLM directly?

■ **Teacher Analogy:** Agar aapko maths mein doubt hai — Scenario 1: Teacher ko puri book de do "Algebra mein doubt hai." vs Scenario 2: "Sir Page 155 mein doubt hai." Scenario 2 mein teacher jaldi aur better response dega! Isliye hum sirf 2 relevant pages LLM ko dete hain — whole book nahi. Reasons: (1) Computationally bahut expensive, (2) LLM ka context window limited hota hai, (3) Results bhi utne accurate nahi aate.

■ 3 Big Challenges in Building This System:

Challenge 1: NLU + Text Generation	Brain banana — query samjhe aur context-aware response de. Yeh problem 2017 ke Transformers paper ne solve ki → BERT + GPT → LLMs. Solution: LLM use karo
Challenge 2: Computational Cost	LLMs bahut heavy hain (100GB+). Inhe apne server pe run karna bahut costly aur complex hai. Solution: LLM APIs (OpenAI, Anthropic, Google) — pay per use
Challenge 3: Orchestration	5-6 different components hain: S3, Text Splitter, Embedding Model, Vector DB, LLM. Inhe milake ek pipeline mein run karna + har stage ke output ko next stage ka input banana — bahut complex. Solution: LangChain! ■

■ LangChain Use Cases — What Can You Build?

#	Use Case	Description / Example
1	Conversational Chatbots	Company-level chatbots jo customer queries handle karte hain. First layer of communication → chatbot → agar resolve na ho toh human agent.
2	AI Knowledge Assistant	Chatbot + aapke own data pe trained/access. Example: CampusX course mein integrated chatbot jahan student lecture dekhte waqt doubt pooch sake.
3	AI Agents	"Chatbots on steroids" — sirf baat nahi karte, kaam bhi karte hain. Example: MakeMyTrip pe senior citizen bole "Book cheapest Delhi-Shimla flight 24 Jan" → Agent API hit karke booking bhi kar dega!
4	Workflow Automation	Personal/professional/company-level workflows ko automate karna. LLM + LangChain ki help se.
5	Summarization & Research Helper	Private company data ya badi books jo ChatGPT pe upload nahi ho sakti (context length / privacy) — apna private tool banao.

■ LangChain Alternatives

Framework	Details	Status
LangChain	Most popular, comprehensive, huge community	Most widely used ■
LlamaIndex	Thoda zyada focused on data/documents, CampusX pe course available	Very popular, comparable to LangChain
Haystack	Similar kind of platform/library, ML-heavy organizations use karte hain	Popular in enterprise

■ **Key Point:** LangChain akela framework nahi hai — alternatives bhi hain. Choice depend karta hai pricing, use case, aur team preference pe.

■ VIDEO 2 — LangChain Components (6 Core Building Blocks)

■ **Goal:** LangChain mein 6 different components hote hain. Agar aapne yeh 6 components padh liye — trust me — aapko LangChain ka majority of concepts samajh mein aa jayenge!

#	Component	One-Line Purpose
1	■ Models	AI models (LLMs + Embedding Models) ke saath communicate karne ka standardized interface
2	■ Prompts	LLM ko diye jaane wale inputs design karne ke tools — templates, few-shot, dynamic prompts
3	■■ Chains	Multiple components ko ek pipeline mein jodhna — ek ka output dusre ka input banta hai automatically
4	■ Memory	Conversation history store karna — chatbot ko pichle messages yaad rehte hain
5	■ Indexes	External data (PDFs, databases) ko LLM ke liye accessible banana — RAG applications ka base
6	■ Agents	"Chatbots on steroids" — reasoning + tools ka access = real-world actions perform kar sakte hain

■ Component 1: Models

Background: The Two Big Problems of NLP (Before LLMs)

- Problem 1: NLU — Natural Language Understanding
 - "Hay can you check my email now" → chatbot samjhe ki user kya maang raha hai
 - Semantic meaning, intent, context — samajhna bahut mushkil tha
- Problem 2: Context-Aware Text Generation
 - Samjhne ke baad relevant, context-aware reply karna
 - Alag topic pe na jaaye, history yaad rahe — yeh bhi ek bada challenge tha

■ **LLMs ne yeh dono problems ek saath solve kar di!** LLMs ko poore internet ke data pe train kiya gaya — isliye unke paas NLU ki bhi capability hai aur context-aware text generation ki bhi. Yahi wajah hai ki ChatGPT itna successful hai.

Third Problem: Standardization (Different API formats)

■■ **Problem:** OpenAI ka API code alag hota hai, Anthropic (Claude) ka alag, Google Gemini ka alag. Agar aapko ek app mein dono use karne hain ya switch karna hai — to poora code change karna padega! Bahut mehnat ka kaam.

■ **LangChain ka Model Component Solution:** Ek common, standardized interface provide karta hai. Sirf 2 lines of code change karo — ek import line + ek model name line — aur aap OpenAI se Claude ya Gemini pe switch ho sakte ho! Response ka structure bhi same rehta hai.

■ Two Types of Models in LangChain:

Type	Input	Output	Use Case	Examples
Language Models (LLMs)	Text (prompt)	Text (response)	Chatbots, QA, summarization, agents	GPT-4, Claude Sonnet, Gemini
Embedding Models	Text	Vector (numbers) = Embeddings	Semantic search, RAG applications, similarity calculation	OpenAI text-embedding-3-large, HuggingFace sentence-transformers

Model Source	Type	Examples in LangChain	Cost
Closed Source (Paid)	Commercial APIs	ChatOpenAI, ChatAnthropic, ChatGoogleGenerativeAI, ChatAzureOpenAI, ChatBedrock (AWS)	Pay per token
Open Source (Free)	Self-hosted / HuggingFace	ChatHuggingFace, ChatOllama — download locally aur run karo	Free (compute cost only)

■ Component 2: Prompts

Prompt (Definition)

Jab aap kisi LLM se interact karte ho, to aap jo message/input bhejte ho — wahi PROMPT hai. "Write a 5-line poem on cricket" → yeh ek prompt hai. Prompts are the instructions you give to LLMs.

Static vs Dynamic Prompts:

• STATIC PROMPTS

- User khud poora prompt type karta hai
- Problem: Inconsistent UX — har user alag-alag likhega
- LLM output highly variable ho jaata hai
- Good for: Simple experiments only
- Example: User types full sentence every time

• DYNAMIC PROMPTS (PromptTemplate)

- Developer ek template banata hai pehle se
- User sirf specific values fill karta hai (placeholders)
- Consistent, professional, controlled output
- LangChain mein PromptTemplate class use karo
- Example: Template fixed, sirf {paper_name} change hota hai

■ PromptTemplate Benefits (3 Key Advantages):

Benefit 1: Input Validation	Agar template mein "paper_name" ka placeholder hai aur aap wo nahi dete → LangChain error throw karta hai. Debugging aasaan ho jaati hai.
Benefit 2: Partial Prompts	Kuch values pehle fill karo, kuch baad mein. Flexible! Intermediate state possible hai.
Benefit 3: LangChain Ecosystem Compatibility	PromptTemplate ko aap directly Chains mein use kar sakte ho. f-strings use karoge to chain mein nahi daal paoge. PromptTemplate = LangChain ke saath best integration.

■ ChatPromptTemplate — Multi-turn Conversations:

■ **3 Types of Messages in Chat:** (1) **System Message** — LLM ko role/context deta hai. "You are a helpful customer support agent." Background instruction — user nahi dekhta. (2) **Human Message (HumanMessage)** — User ka message. "Can I get a refund for Order #12345?" (3) **AI Message (AIMessage)** — LLM ka response. "Your refund has been initiated."

■ **MessagePlaceholder** — Puri chat history ke liye ek placeholder. Jab user naya message bhejta hai — purani saari history is placeholder ki jagah automatically aa jaati hai. LLM ko context milta hai aur wo samajhta hai ki conversation kahan hai.

■ LLM Temperature Parameter (Important Correction):

Temperature Value	Behavior	Use Case
0 (Zero)	Same input → ALWAYS same output (deterministic)	When you need consistent, reproducible results (e.g., data extraction, fact-based QA)
0.5 (Medium)	Slightly creative, small variation	Balanced apps — some creativity, some consistency
1.5–2 (High)	Very creative, DIFFERENT output each time from same input	Creative writing, brainstorming, poetry, diverse responses

■■ **Common Misconception:** Temperature "0 se 2 ke beech" ki baat toh sahi hai, but yeh "deterministic vs creative" distinction bahut important hai — yeh ek student ne comment mein point out kiya aur instructor ne correct kiya!

■ Prompt Engineering Techniques (Overview):

Technique	Description	LangChain Support
Zero-Shot Prompting	Seedha question — koi example nahi dete	■ Default behavior

Few-Shot Prompting	Pehle kuch examples dikhao, phir question poochho. Example: Customer support ticket classification — 3-4 examples de ke batao ki billing issue kya hai, technical issue kya hai, phir naya ticket classify karo.	■ FewShotPromptTemplate
Role-Based Prompting	"You are an experienced doctor" → LLM doctor ki tarah respond karta hai. Dynamic role + dynamic topic = powerful template!	■ SystemMessage in ChatPromptTemplate
Chain of Thought (CoT)	Complex reasoning ke liye step-by-step sochne ke liye force karo. AI Agents mein heavily used.	■ In Agents component
Multimodal Prompts	Text ke alawa image/audio/video bhi bhej sakte ho. Example: Image upload + "Singer kaun hai?"	■ But mostly text-based today

■ Building a Simple Chatbot (Practical Demo from Video 4):

Memory-less Chatbot Problem: Basic chatbot mein ek problem hoti hai — woh previous messages "bhool jaata hai"! Example: "2 aur 0 mein kya bada hai?" → AI: "2". Ab aap bolo "Bade number ko 10 se multiply karo" → AI: "Let x be bigger number, 10x" ■ — kyunki usse yaad hi nahi ki 2 bada tha! **Solution:** Chat history maintain karo ek list mein. Har nayi query ke saath puri history LLM ko bhejo. LLM tab context samajhega aur correct response dega.

■ Component 3: Chains

Chains (Definition)

LangChain ka woh component jisme aap multiple steps ko ek pipeline mein connect karte ho. **Ek stage ka output automatically dusre stage ka input ban jaata hai** — manually kuch nahi karna. LangChain ka naam bhi "chain" se hi aaya hai!

Without Chains vs With Chains:

• Manual Pipeline (No Chains)

- Step 1: User se input lo
- Step 2: LLM-1 ko call karo (translate)
- Step 3: Output manually uthao
- Step 4: LLM-2 ko call karo (summarize)
- Step 5: Final output lo — MANUALLY code likhna padta hai

• Using Chains

- `template | model = chain` (LCEL syntax)
- `chain.invoke({"input": user_text})`
- Sab kuch automatically execute hota hai
- No manual passing of outputs/inputs
- Cleaner, shorter, more Pythonic code!

■ LCEL — LangChain Expression Language (Chain Syntax):

■ **LCEL Pipe Syntax:** `chain = template | model` — Pipe operator (`|`) se components ko join karo. Template ka output → Model ka input. `chain.invoke({"input": value})` — sab kuch ek baar mein execute! Aur complex: `chain = prompt | model | output_parser`

Types of Chains:

Chain Type	Description	Use Case
Sequential Chain	$A \rightarrow B \rightarrow C \rightarrow D$ (ek ke baad ek)	Translation → Summarization pipeline
Parallel Chain	$A \rightarrow (B \text{ aur } C \text{ saath mein}) \rightarrow D$	Multiple analyses simultaneously
Conditional Chain	Condition ke basis pe different path lena	If query is about billing → billing bot, else → general bot

■ Component 4: Memory

Memory (Definition)

LangChain ka Memory component conversation history ko track karta hai. Bina memory ke har query independent hoti hai — LLM ko koi context nahi milta. Memory se chatbot "yaaddasht" rakh sakta hai.

Types of Memory:

Memory Type	How it Works	Best For
ConversationBufferMemory	Puri saari history store karta hai as-is	Short conversations, simple chatbots
ConversationBufferWindowMemory	Sirf last K messages yaad rakhta hai (K=5 etc.)	Medium conversations, memory cost control
ConversationSummaryMemory	Old messages ka summary banata hai, full history discard karta hai	Long conversations, token limit management
ConversationSummaryBufferMemory	Recent messages full + old messages summarized	Best of both worlds — production apps

■ Component 5: Indexes (Retrievers / RAG)

Indexes / Retrievers

Yeh component external data ko LLM ke liye accessible banata hai. Iska use RAG (Retrieval Augmented Generation) applications mein hota hai — jaise ki "Chat with your PDF/Database".

Sub-Component	Role
Document Loaders	PDF, Word, web pages, databases se data load karo
Text Splitters	Bade documents ko manageable chunks mein todo

Embedding Models	Chunks ko vector embeddings mein convert karo
Vector Stores	Embeddings ko store karo (Pinecone, FAISS, Chroma)
Retrievers	Query ke basis pe most relevant chunks retrieve karo

■ Component 6: Agents (Chatbots on Steroids!)

AI Agents (Definition)

"Agents are chatbots on steroids." Chatbot sirf baat kar sakta hai, lekin AI Agent kuch **kaam bhi karke de sakta hai!** Yeh aaj ke AI world ka next big thing hai.

Chatbot vs AI Agent — Key Difference:

• Chatbot • Sirf baat kar sakta hai • "Best summer destinations India?" → Shimla, Manali bata dega • Koi action nahi le sakta • No reasoning, no tool access • Just NLU + Text Generation	• AI Agent • Baat bhi karta hai + KAAM bhi karta hai! • "Book cheapest Delhi-Shimla flight 24 Jan" → Actually API hit karke ticket book kar dega! • 2 extra capabilities: • ■ Reasoning Capability (Chain of Thought) • ■ Access to Tools (APIs, calculators, databases)
---	---

■ How AI Agent Works — Step by Step Example:

Setup: AI Agent ke paas 2 tools hain — (1) Calculator tool, (2) Weather API tool **User Query:** "Can you multiply today's temperature of Delhi by 3?" **Agent Reasoning (Chain of Thought):** → Step 1: "Mujhe Delhi ka aaj ka temperature chahiye" → Weather API tool call karo → Result: 25°C → Step 2: "Ab 25×3 karna hai" → Calculator tool call karo with (25, 3, multiply) → Result: 75 → Step 3: Final answer: 75 user ko do!
Key Insight: Agent ne task ko steps mein break down kiya, sahi tools use kiye, aur final answer nikala. Yahi reasoning + tools = AI Agent magic!

■ **Future of AI Agents:** "Sari badi companies aur AI researchers is topic pe converge kar rahe hain. Agle 1-1.5 saal mein is front pe bahut progress hone wali hai." — Instructor Nitesh

■ VIDEO 3 — Models Component: Deep Dive + Coding

■ Video Goal: Pehle 2 videos theory the — ab is video mein actual code likhenge aur models ke saath interact karenge! Language Models + Embedding Models — dono types cover honge.

■■ Part 1: Language Models in LangChain

Closed-Source Models (Paid APIs):

Provider	LangChain Class	Model Example	API Key Required
OpenAI	ChatOpenAI	gpt-4, gpt-3.5-turbo	OPENAI_API_KEY
Anthropic	ChatAnthropic	claude-sonnet-4, claude-opus	ANTHROPIC_API_KEY
Google	ChatGoogleGenerativeAI	gemini-pro, gemini-flash	GOOGLE_API_KEY
AWS Bedrock	ChatBedrock	Llama, Titan, etc.	AWS credentials
Azure OpenAI	ChatAzureOpenAI	GPT models on Azure	Azure credentials

Open-Source Models (Free, Local):

Source	LangChain Class	How to Use	Cost
HuggingFace	ChatHuggingFace	HuggingFace Hub se model download karo, locally ya Hub API se run karo	Free (compute cost only)
Ollama	ChatOllama	Ollama se local machine pe LLM run karo (llama3, mistral, etc.)	Free, fully local

■ Key LLM Parameters to Remember:

Parameter	Description	Values/Range
model	Konsa model use karna hai	"gpt-4", "claude-3-sonnet", "gemini-pro" etc.
temperature	Output ki creativity/randomness. 0 = deterministic, 2 = very creative	0 to 2 (float)
max_tokens	Response mein maximum tokens (words) kitne aayenge	Integer (e.g., 1000)
top_p	Nucleus sampling — probability threshold for token selection	0 to 1 (float)

■ Code Pattern — Interacting with LLMs in LangChain:

```
# Install: pip install langchain langchain-openai python-dotenv
from langchain_openai import ChatOpenAI
from dotenv import load_dotenv
load_dotenv() # .env file se OPENAI_API_KEY load karo

# Model create karo
model = ChatOpenAI(model="gpt-4", temperature=0)

# Simple invoke
result = model.invoke("Write a 5-line poem on cricket")
print(result.content) # result ek AIMessage object hai

# Switch to Anthropic – bas 2 lines change!
from langchain_anthropic import ChatAnthropic
model = ChatAnthropic(model="claude-sonnet-4-6") # Same invoke() call!
```

■ Part 2: Embedding Models in LangChain

Embeddings (Definition)

Text input leke uska **vector (numbers ki list)** return karte hain. Yeh numbers text ka semantic meaning represent karte hain. Jab aap "What is the capital of India?" dete ho — embedding model 300 ya 1000+ numbers return karta hai jo is sentence ka meaning represent karte hain.

Available Embedding Models in LangChain:

Provider	Class	Model	Dimensions	Notes
OpenAI	OpenAIEmbeddings	text-embedding-3-large, text-embedding-3-small	3072, 1536	Most popular, paid API
HuggingFace	HuggingFaceEmbeddings	all-MiniLM-L6-v2, etc.	384, 768	Free, runs locally, download from Hugging Face Hub
Google	GoogleGenerativeAIEmbeddings	embedding-001	768	Paid API

Two Types of Embedding Operations:

- **embed_documents(list_of_texts)**

- Multiple texts ke embeddings ek saath banao
- Returns: List of vectors (2D list)
- Use: Documents/corpus ko embed karne ke liye
- Example: 5 cricketers ke paragraphs → 5 vectors

- **embed_query(single_text)**

- Ek single query/text ka embedding banao
- Returns: Single vector (1D list)
- Use: User ki query ko embed karne ke liye
- Example: "Tell me about Virat Kohli" → 1 vector

■ Document Similarity App — Practical Example:

Problem: 5 documents hain (5 cricketers ke baare mein). User query: "Tell me about Virat Kohli." Kaunsa document most relevant hai? **Solution Steps:** 1. documents = [virat_text, bumrah_text, rohit_text, ...] 2. doc_embeddings = embedding.embed_documents(documents) → 5 vectors 3. query_embedding = embedding.embed_query(user_query) → 1 vector 4. scores = cosine_similarity([query_embedding], doc_embeddings) → 5 similarity scores 5. best_idx = argmax(scores) → highest score wala document retrieve karo 6. Output: "Virat Kohli is an Indian cricketer... (Similarity: 0.66)" **Result:** "Tell me about Bumrah" → Jasprit Bumrah wala document (66% similarity), correct!

■ **Why Vector Databases? Jab aap real app banate ho — har baar naya query aane pe embeddings dobara generate karna expensive hai! Solution: Document embeddings ek baar generate karo aur Vector Database mein store karo (Pinecone, FAISS, ChromaDB). Naya query aaye → sirf query embed karo → DB se similarity search karo. Yahi proper RAG architecture hai!**

■ VIDEO 4 — Prompts Component: Deep Dive + Coding

■ PromptTemplate — Dynamic Prompts

Research Assistant Tool — Complete Example:

Use Case: Ek research assistant tool banao jahan user koi bhi research paper ka summary le sake — apne preferred style aur length mein. **Template Design:** "Please summarize the research paper titled {paper_input} with the following specifications: explanation style: {style_input}, explanation length: {length_input}. Include relevant mathematical equations if present. Explain using simple intuitive code snippets where applicable. Also add analogies where relatable. If information is unavailable, put 'Insufficient Information Available'. Ensure summary is clear, accurate, aligned with provided style and length." **User provides:** (1) Paper name, (2) Style (simple/technical/ELI5), (3) Length (5 lines / 1 page) **Developer controls:** Rest of the template — consistent quality every time!

■ PromptTemplate Code Pattern:

```
from langchain_core.prompts import PromptTemplate, ChatPromptTemplate
from langchain_openai import ChatOpenAI

# PromptTemplate (simple, single prompt)
template = PromptTemplate(
    template="Summarize {paper_input} in {style} style, {length} length.",
    input_variables=["paper_input", "style", "length"]
)

# Use with a Chain (LCEL)
model = ChatOpenAI()
chain = template | model
result = chain.invoke({
    "paper_input": "Attention is All You Need",
    "style": "simple",
    "length": "5 lines"
})
print(result.content)
```

■ ChatPromptTemplate — Conversation Style

```
from langchain_core.prompts import ChatPromptTemplate, MessagesPlaceholder

# System + History + Human message template
chat_template = ChatPromptTemplate.from_messages([
```

```

("system", "You are a helpful {role}. Answer about {topic}."),
MessagesPlaceholder(variable_name="chat_history"), # Past messages
("human", "{user_query}"),
])

# Invoke with context
result = chat_template.invoke({
    "role": "customer support agent",
    "topic": "refunds",
    "chat_history": loaded_history, # Previous messages list
    "user_query": "Where is my refund?"
})

```

■ Real-World Chatbot with Context (MessagePlaceholder):

Scenario: Customer kal refund request kiya. Aaj phir aaya aur pucha "Where is my refund?" **Without MessagePlaceholder:** LLM ko pata hi nahi kya hua tha — context missing! **With MessagePlaceholder:** 1. Chat history DB/file mein save hai: [HumanMessage("I want refund for Order 12345"), AIMessage("Refund initiated, 5 business days")] 2. MessagePlaceholder("chat_history") — yeh history automatically template mein insert ho jaati hai 3. LLM ko full context milta hai — "Where is my refund?" ko samajh ke proper answer deta hai

■ Building a Streamlit UI for LLM App

Streamlit: Python ki ek library jo aapko bahut kum code mein web UI banane deti hai. Sirf Python — no HTML/CSS/JS needed! **Run karo:** `streamlit run app.py` → automatically browser mein open ho jaata hai **Key Streamlit Functions:** • `st.header("Title")` — heading • `st.text_input("Enter prompt")` → user ka input • `st.button("Summarize")` → click event • `st.write(result)` → output display

■ VIDEO 5 — Structured Output: JSON, Pydantic, JSON Schema

■ What is Structured Output?

• Unstructured Output

- LLM ka normal text response
- "New Delhi is the capital of India."
- "Here is a suggested itinerary: Morning: Visit Eiffel Tower. Afternoon: Visit a museum..."
- Plain text — no schema, no structure
- Hard to parse programmatically

• Structured Output

- LLM response in a defined DATA FORMAT
- JSON: [{"time": "Morning", "activity": "Visit Eiffel Tower"}, ...]
- Defined schema — predictable keys + value types
- Easy to store in DB, call APIs, process in code
- Machine-readable, integration-friendly

Structured Output (Definition)

In LangChain, **structured output** refers to the practice of **having language models return responses in a well-defined data format**. This makes the model output easier to parse and work with programmatically — allowing integration with other systems like databases, APIs, and agents.

■ 3 Key Use Cases for Structured Output

#	Use Case	Example	How Structured Output Helps
1	Data Extraction	Job portal (Naukri.com) — resume parse karo	Resume text → LLM → JSON {name, company, marks, college} → directly insert in DB
2	Building APIs	Amazon product reviews — unstructured text se structured info nikalo	Review text → LLM → {key_themes: [], summary: "", sentiment: "positive", pros: [], cons: []} → Flask/FastAPI endpoint
3	Building Agents	Agent ko tools call karni hoti hain → function calling	Agent → structured JSON → function/tool call → execute action

■ 3 Ways to Define Schema for Structured Output

Method 1: TypedDict (Simplest)

```
from typing_extensions import TypedDict, Annotated
from typing import Optional

class Review(TypedDict):
    key_themes: Annotated[list[str], "Key themes discussed in review"]
```



```

summary: Annotated[str, "Brief summary of the review"]
sentiment: Annotated[str, "Positive, Negative, or Neutral"]
pros: Annotated[Optional[list[str]], "List of pros (optional)"]
cons: Annotated[Optional[list[str]], "List of cons (optional)"]

# Use with LLM
structured_model = model.with_structured_output(Review)
result = structured_model.invoke("Review: Great battery life but camera is average.")
# result is a Python DICTIONARY (TypedDict object)

```

Method 2: Pydantic (Recommended for Production)

```

from pydantic import BaseModel, Field
from typing import Optional, List, Literal

class Review(BaseModel):
    key_themes: List[str] = Field(description="Key themes discussed")
    summary: str = Field(description="Brief summary")
    sentiment: Literal["pos", "neg", "neutral"] = Field(description="Overall sentiment")
    pros: Optional[List[str]] = Field(default=None, description="Pros mentioned")
    cons: Optional[List[str]] = Field(default=None, description="Cons mentioned")
    product_name: Optional[str] = Field(default=None)

# Use with LLM (same as TypedDict)
structured_model = model.with_structured_output(Review)
result = structured_model.invoke("Review text here...")
# result is a PYDANTIC OBJECT – result.summary, result.sentiment, etc.

```

Method 3: JSON Schema (Cross-Language Compatibility)

```

json_schema = {
    "title": "ReviewType",
    "type": "object",
    "properties": {
        "key_themes": {"type": "array", "items": {"type": "string"},
        "description": "Key themes"},
        "summary": {"type": "string", "description": "Brief summary"},
        "sentiment": {"enum": ["pos", "neg", "neutral"]},
        "pros": {"anyOf": [{"type": "array", "items": {"type": "string"}}, {"type": "null"}]},
        "name": {"anyOf": [{"type": "string"}, {"type": "null"}]}
    },
    "required": ["key_themes", "summary", "sentiment"]
}

structured_model = model.with_structured_output(json_schema)

```

```
result = structured_model.invoke("Review text...") # Returns: Python DICTIONARY
```

■ Comparison: Which Method to Use?

Feature	TypedDict	Pydantic	JSON Schema
Output Type	Python Dictionary	Pydantic Object	Python Dictionary
Data Validation	■ No	■ Yes — auto-validates types	■ Limited
Default Values	■ No	■ Yes — set defaults for optional fields	■ No
Auto Type Conversion	■ No	■ Yes — "42" → 42 automatically	■ No
Cross-Language Share	■ Python only	■ Python only	■ JSON schema = universal
When to Use?	Simple Python projects, quick prototyping	Production apps — most recommended ■	When sharing schema across teams/languages

■ **Instructor Recommendation:** "Hum mostly Pydantic ke upar zyada focus karenge — yeh hamara go-to format hoga jab bhi schema define karna ho. Python ecosystem mein sabse powerful aur flexible option hai."

■■ Two Methods for Getting Structured Output

Method	When to Use	How it Works
JSON Mode	Claude, Gemini, aur most non-OpenAI models ke saath. Tab use karo jab JSON format mein output chahiye (most common case).	model.with_structured_output(schema, method="json_mode")
Function Calling	OpenAI models ke saath (default). Tab use karo jab structured output isliye chahiye kyunki aap koi function/tool call karna chahte ho — AI Agents mein common.	model.with_structured_output(schema, method="function_calling")

■■ **Important:** Kuch open-source models (jaise TinyLlama on HuggingFace) structured output support nahi karte — na JSON mode mein, na function calling mein. Aise models ke liye aapko manually output parsers likhne padte hain (next video topic).

■ COMPREHENSIVE QUICK REFERENCE GUIDE

■ LangChain Installation & Setup

```
# Core packages
pip install langchain langchain-core langchain-community
pip install langchain-openai langchain-anthropic langchain-google-genai
pip install python-dotenv # For API key management

# .env file mein rakhein (never hardcode!)
OPENAI_API_KEY=sk-...
ANTHROPIC_API_KEY=sk-ant-...
GOOGLE_API_KEY=...

# Python mein load karo
from dotenv import load_dotenv
load_dotenv() # Automatically .env file padhega
```

■ Complete LangChain Cheat Sheet

Task	LangChain Code	Notes
Create ChatOpenAI model	<code>model = ChatOpenAI(model="gpt-4", temperature=0)</code>	Default temperature=0.7
Create Anthropic model	<code>model = ChatAnthropic(model="claude-sonnet-4-6")</code>	Switch with minimal code change
Invoke model	<code>result = model.invoke("prompt") → result.content</code>	Returns AIMessage object
PromptTemplate	<code>tpl = PromptTemplate(template="..{var}..", input_variables=["var"])</code>	For single-turn prompts
ChatPromptTemplate	<code>ChatPromptTemplate.from_messages([("system", " "..), ("human", " "..)])</code>	For multi-turn chat
Create a chain	<code>chain = prompt model</code>	LCEL pipe syntax — PromptTemplate+model
Invoke chain	<code>result = chain.invoke({"var": "value"})</code>	Dictionary of template variables
Embed documents	<code>emb.embed_documents(["text1", "text2"])</code>	Returns list of vectors
Embed query	<code>emb.embed_query("query text")</code>	Returns single vector

Structured output	<code>struct_model</code> <code>model.with_structured_output(Schema)</code>	=	Schema TypedDict/Pydantic/JSON	=
Cosine similarity	<code>cosine_similarity([q_emb], doc_embs)</code>		from sklearn.metrics.pairwise import cosine_similarity	

■ LangChain Concepts Summary Table

Concept	What it is	Why it matters
Model Component	Standardized interface for all AI models	Switch between OpenAI/Claude/Gemini with 2 lines of code
Language Model	Text in → Text out (LLM)	Chatbots, agents, QA, summarization
Embedding Model	Text in → Vector out (numbers)	Semantic search, RAG, similarity
Vector/Embedding	Text as numerical representation (e.g., 300 numbers)	Math operations on semantic meaning
Semantic Search	Meaning-based search using vectors + cosine similarity	Find relevant documents, power RAG apps
RAG	Retrieval Augmented Generation — LLM + external data	"Chat with your PDF/Database" applications
PromptTemplate	Template with placeholders — dynamic prompts	Consistent, reusable, chain-compatible prompts
ChatPromptTemplate	Multi-message template (System + Human + AI)	Chatbot applications, role-based interactions
MessagePlaceholder	Placeholder for chat history in templates	Context-aware chatbots with memory
Chain (LCEL)	Pipeline of components — output flows automatically	Complex multi-step LLM workflows
AI Agent	LLM + Reasoning + Tools = autonomous task execution	Book flights, query DBs, send emails automatically
Structured Output	LLM response in JSON/Pydantic format	Integration with DBs, APIs, agents, downstream systems
Pydantic	Python data validation library for schemas	Best way to define structured output schema in Python

Function Calling	Structured output method for tool/function calls	Core mechanism of AI Agents
Temperature=0	Deterministic output — same input, same output always	Production apps needing consistency
Temperature=1.5	Creative output — same input, different output each time	Creative writing, brainstorming apps

■■ Series Roadmap — What's Coming Next

Video #	Topic	Key Content
1 (Done ■)	Introduction	LangChain kya hai, kyu chahiye, use cases, alternatives
2 (Done ■)	Components Overview	6 components ka high-level intro + examples
3 (Done ■)	Models Deep Dive	Language + Embedding models, code, similarity app
4 (Done ■)	Prompts Deep Dive	PromptTemplate, ChatPromptTemplate, Streamlit app, chatbot
5 (Done ■)	Structured Output	TypedDict, Pydantic, JSON Schema, use cases
6 (Upcoming)	Output Parsers	Models jo structured output support nahi karte — manual parsing
7+	Chains Deep Dive	LCEL, sequential/parallel/conditional chains, real apps
8+	Memory Deep Dive	All memory types code mein
9+	Indexes/RAG	Vector stores, retrievers, full RAG pipeline build
10+	Agents Deep Dive	ReAct, CoT, tools, full AI agent banana

■ **Final Note from Instructor Nitesh: "LLM-based applications ka boom aane wala hai — jaise websites ka aaya tha, jaise apps ka aaya tha. Aur wahan par LangChain ek bahut key role play karne wala hai. Is playlist ke end tak aap ek full AI Agent build kar ke dikhayenge! ■"**